

UNIVERSIDADE PAULISTA – UNIP EaD

Projeto Integrado Multidisciplinar

Curso Superior de Tecnologia em
Análise e Desenvolvimento de Sistemas

Autores	R.A
Yasmine Lima Santana	2526592
Luan Lopes Bezerra	2503460
Geovanna Félix Monteiro	2521699
João Pedro Carneiro da Silva	2540975
Vitória Freitas	2523279
Sofia de Castro Ongaro	2528018

**Ambiente virtual responsivo para eventos acadêmicos
inclusivos em TI**

Santana de Parnaíba, SP

2025

Autores	R.A
Yasmine Lima Santana	2526592
Luan Lopes Bezerra	2503460
Geovanna Félix Monteiro	2521699
João Pedro Carneiro da Silva	2540975
Vitória Freitas	2523279
Sofia de Castro Ongaro	2528018

Ambiente virtual responsivo para eventos acadêmicos inclusivos em TI

Projeto Integrado Multidisciplinar em
Análise e Desenvolvimento de Sistemas

Projeto Integrado Multidisciplinar para obtenção do título de tecnólogo em
Análise e Desenvolvimento de Sistemas, apresentado
à Universidade Paulista – UNIP EaD.

Santana de Parnaíba, SP

2025

Resumo

O projeto "Hello World Conference" teve como objetivo criar uma plataforma web para organização e divulgação de eventos acadêmicos na área de Tecnologia da Informação. A proposta integrou desenvolvimento técnico com aspectos humanos, como comunicação, liderança e inclusão.

A plataforma foi desenvolvida com um design moderno e responsivo, permitindo que os usuários visualizem eventos, se inscrevam, interajam por meio de comentários e emitam certificados de forma simples e intuitiva. No back-end, foi utilizada uma API em C# para gerenciar todas as funcionalidades do sistema, garantindo organização e eficiência.

Além disso, foi elaborado um plano de comunicação para facilitar o contato entre participantes e organizadores, incluindo mensagens automatizadas e modelos visuais atrativos, como e-mails em HTML. A liderança e a negociação também foram importantes para organizar o trabalho em equipe e lidar com possíveis conflitos durante o desenvolvimento.

Um dos principais diferenciais do projeto foi a preocupação com a acessibilidade, com a inclusão de recursos em Libras, como vídeos, glossários e suporte ao usuário surdo, garantindo uma experiência mais inclusiva.

Dessa forma, o projeto une tecnologia, organização e inclusão, oferecendo uma solução completa e acessível para eventos acadêmicos.

Palavras-chave: Plataforma Web; Eventos Acadêmicos; Acessibilidade; Libras; Comunicação Organizacional.

Abstract

The "Hello World Conference" project aimed to develop a web platform for the organization and promotion of academic events in the field of Information Technology. The proposal integrated technical development with human aspects, such as communication, leadership, and inclusion.

The platform was developed with a modern and responsive design, allowing users to view events, register, interact through comments, and issue certificates in a simple and intuitive way. On the back-end, a C# API was used to manage all system functionalities, ensuring organization and efficiency.

In addition, a communication plan was developed to facilitate interaction between participants and organizers, including automated messages and visually appealing formats, such as HTML emails. Leadership and negotiation also played an important role in organizing teamwork and handling potential conflicts during development.

One of the main highlights of the project was its focus on accessibility, including features in Brazilian Sign Language (Libras), such as videos, glossaries, and support for deaf users, ensuring a more inclusive experience.

Thus, the project combines technology, organization, and inclusion, offering a complete and accessible solution for academic events.

Keywords: Web Platform; Academic Events; Accessibility; Libras; Organizational Communication.

Sumário

Introdução	6
1. Web Responsivo	7
1.1 Desenvolvimento da Interface do Projeto "Hello World Conference"	7
2. Código Fonte Web Responsivo	8
2.1 HTML e CSS da página de eventos, página inicial	8
2.2 HTML e CSS da página de comentários dos eventos	15
2.3 HTML e CSS da página de login e cadastro	23
2.4 HTML e CSS da página do usuário e da emissão de certificado	36
3. Programação orientada a objetos com c#	56
3.1 Construção de API Web com ASP.NET Core para o desenvolvimento da aplicação web	56
3.2 Pasta com os Models do sistema	59
3.3 Pasta que contém uma classe do sistema	61
3.4 Pasta com os services do sistema	62
3.5 Pasta com os Controllers do Sistema	77
3.6 Prints de casos de testes	87
4. Comunicação, Liderança e Negociação no Projeto "Hello World Conference"	89
5. Língua Brasileira de Sinais – Libras	94
5.1 Glossário de Termos Técnicos em Libras	94
5.2 Storyboard dos Recursos de Acessibilidade	94
5.3 Storyboard de Orientação, Navegação e Apoio ao Participante Surdo .	95
5.4 Análise Crítica e Perspectiva Bilíngue	95
Conclusão	97
Bibliografias	98

Introdução

Com o avanço da tecnologia, a organização de eventos acadêmicos passou a depender cada vez mais de plataformas digitais eficientes e acessíveis. Nesse cenário, torna-se importante desenvolver soluções que integrem não apenas aspectos técnicos, mas também fatores humanos, como comunicação, liderança e inclusão.

Diante disso, o presente trabalho propõe o desenvolvimento da plataforma "Hello World Conference", um ambiente web responsivo voltado para a organização e divulgação de eventos na área de Tecnologia da Informação. A plataforma permite a realização de inscrições, visualização de programação, interação entre usuários e emissão de certificados de forma prática e intuitiva.

Além dos aspectos técnicos, o projeto também contempla a elaboração de um plano de comunicação, bem como a aplicação de práticas de liderança e negociação no desenvolvimento da solução. Como diferencial, destaca-se a inclusão de recursos em Libras, promovendo acessibilidade e ampliando o acesso de pessoas surdas ao ambiente digital.

Dessa forma, o projeto busca unir tecnologia e inclusão, oferecendo uma solução completa para a gestão de eventos acadêmicos.

1. Web Responsivo

1.1 Desenvolvimento da Interface do Projeto "Hello World Conference":

O projeto consiste em uma plataforma web voltada para o gerenciamento e divulgação de eventos acadêmicos na área de Tecnologia da Informação (TI). O nome escolhido, "Hello World Conference", utiliza uma metáfora universal da programação para estabelecer uma conexão imediata com o público-alvo (estudantes e profissionais de tecnologia), simbolizando o início do aprendizado e a introdução a novos conceitos.

A interface foi construída com base em um moodboard que prioriza a estética "Tech-Modern".

- **Paleta de Cores:** Utilizou-se uma paleta análoga configurada em RGB (otimizada para telas), composta por tons de roxo, azul e verde neon. Psicologicamente, o roxo e o azul evocam inovação, confiança e o ambiente digital futurista, enquanto o verde neon remete às interfaces de terminais e linhas de código.
- **Tipografia e Grafismos:** A escolha de fontes sem serifa e o uso de elementos como chaves { }, slashes / e a estética de "grade" reforçam a semântica da programação no front-end.

O sistema apresenta um fluxo de usuário lógico e funcional, dividido em módulos principais visualizados nas telas:

- **Landing Page e Eventos:** Uma lista clara de eventos (como "Programação em Massa" e "Saga Dev") com chamadas para ação (CTAs) diretas: "Inscreva-se" e "Ir para Comentários".
- **Sistema de Feedback:** Uma seção dedicada a comentários, permitindo a interação social entre os participantes, essencial para o engajamento acadêmico.
- **Área Administrativa e Cadastro:** Formulários intuitivos para registro de novos usuários e criação de eventos, incluindo campos específicos para gestão, como a "Key para emissão de certificado".

- **Gamificação e Certificação:** O uso de uma "Chave de Acesso" para gerar certificados demonstra uma preocupação com a segurança e a validação da participação no evento.

O design optou por um fundo limpo (branco) nas áreas de conteúdo para garantir a legibilidade, contrastando com cabeçalhos e rodapés em tons escuros que delimitam a área de navegação. Elementos visuais abstratos e globos estilizados são usados como "background" para dar profundidade à interface sem distrair o usuário das tarefas principais (login, cadastro e visualização de eventos).

A estruturação do front-end da "Hello World Conference" alinha a estética técnica à funcionalidade necessária para um ambiente acadêmico. A coesão entre a paleta de cores, a nomenclatura e a disposição dos elementos reflete um ambiente de inovação, cumprindo o objetivo proposto de facilitar a organização e a participação em eventos tecnológicos.

2. Código Fonte Web Responsivo

2.1 HTML e CSS da página de eventos, página inicial:

HTML da página:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <title>Document</title>

    <script type="module" src="js/script.js"></script>

    <link rel="stylesheet" href="../global.css">
```

```

    <link rel="stylesheet" href="index.css">

</head>

<body>

    <nav>

        <p><a
href="../../Login_Cadastro/Login/login.html">Login/Cadastro</a></p>

        <p><a href="index.html">Eventos Registrados</a></p>

    </nav>

    <main class="events-container">

        <h1>Eventos Tecnológicos em São Paulo</h1>

        <!-- Eventos inseridos dinamicamente aqui -->

    </main>

    <footer>

        <p>Site para registro e criação de eventos tecnológicos.</p>

        <p>Feito por: </p>

        <p>Vitória Freitas</p>

    </footer>

</body>

</html>

```

CSS da página:

```
body {
```

```

background-image: url("../img/hello_world_white.png");

background-position: left bottom left bottom;

background-repeat: no-repeat;

background-attachment: fixed;

min-height: 100vh;

padding-bottom: 2rem;
}

h1{

margin-top: 1rem;

}

.events-container {

display: flex;

flex-direction: column;

gap: 1rem;

max-width: 800px;

margin: 2rem auto;

padding: 1rem;

z-index: 1;

}

.event-container {

background-color: rgba(255, 255, 255, 0.74);

border-radius: 8px;

```

```

padding: 1rem;

box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.events-container h1 {

    text-align: center;

    color: #333;

    margin-bottom: 2rem;
}

.event-container button:hover {

    background-color: #555;
}

```

Função que insere dinamicamente os eventos com JavaScript:

```

async function loadEventsInTheScreen() {

    const events = await
    get_func("https://localhost:7120/api/Events/getEvents")

    events.map((events) => {

        const div = document.createElement("div")

        const pMessageToUser = document.createElement("p")

        const h2Title = document.createElement("h2")

        const pDesc = document.createElement("p")

        const pData = document.createElement("p")
    })
}

```

```
const subscribeBtn = document.createElement("button")

const goToComments = document.createElement("button")

const date = new Date(events.date)

h2Title.textContent = events.title

pDesc.textContent = events.description

pData.textContent = date.toLocaleDateString("en-GB")

subscribeBtn.textContent = "Inscreva-se"

goToComments.textContent = "Ir para os comentários"

div.classList.add("event-container")

div.appendChild(pMessageToUser)

div.appendChild(h2Title)

div.appendChild(pDesc)

div.appendChild(pData)

div.appendChild(subscribeBtn)

div.appendChild(goToComments)

eventsContainer.appendChild(div)
```

```

subscribeBtn.addEventListener("click", async () => {

    const keyLocal = localStorage.getItem("key")

    if(!keyLocal) pMessageToUser.textContent = "Precisa estar
logado para se inscrever ao evento."

    const keyNumber = keyLocal.split("T")[0] + "T" // pega o
valor o da key junto com o T

    const userId = keyLocal.split("T")[1] // pega apenas o id

    if(keyNumber !== key) pMessageToUser.textContent = "Precisa
estar logado para se inscrever ao evento."

    const data = {

        userId: userId,

        eventId: events.id

    }

    const registerToEvent = await UserFetch(data,
"https://localhost:7120/api/Site/registerToEvent")

    pMessageToUser.textContent = registerToEvent === true ?
"Inscrito com sucesso." : "Ocorreu um erro, tente novamente mais
tarde."

```

```

        setTimeout(() => {

            pMessageToUser.textContent = ""

        }, 2000)

    })

    goToComments.addEventListener("click", async () => {

        window.location.assign(`/site/EventPage/comments.html?id=${events.id}`
        )

    })

})

}

```

Print da tela inicial, com todas as partes mostradas juntas:



Print da tela em um celular:



2.2 HTML e CSS da página de comentários dos eventos:

HTML da página:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <title>Document</title>

    <script type="module" src="js/comments.js"></script>

    <link rel="stylesheet" href="../global.css">
```

```

    <link rel="stylesheet" href="comments.css">

</head>

<body>

    <nav>

        <p><a
href="../../Login_Cadastro/Login/login.html">Login/Cadastro</a></p>

        <p><a href="index.html">Eventos Registrados</a></p>

    </nav>

    <main class="comments-main">

        <h1>Comentários</h1>

        <section class="comments-section">

            <!-- Comentários inseridos dinamicamente aqui -->

        </section>

        <p id="user-message"></p>

        <section class="comments-add-section">

            <h3>Adicione um comentário</h3>

            <textarea id="comment"></textarea>

            <button                                class="commentsBtn"
id="commentBtn">Adicionar</button>

```

```
        </section>

    </main>

</body>

</html>
```

CSS da página:

```
h1{

    text-align: center;

    margin-top: 4rem;

    text-decoration: underline;

}

section{

    width: 50%;

    margin: 0 auto;

    margin-bottom: 1rem;

    background-color: #f9f9f9;

    border-radius: 8px;

    padding: 1rem;

    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);

}

.comments-section div{

    border-bottom: 1px solid rgba(145, 142, 142, 0.623);

}
```

```

padding: 0.5rem;
}

.comments-add-section{

    display: flex;

    flex-direction: column;
}

.comments-add-section button{

    width: 30%;

    margin-top: 0.7rem;
}

.comments-add-section h3{

    margin-bottom: 0.7rem;
}

@media screen and (max-width: 900px) {

    .comments-main {

        max-width: 650px;

        margin: 0 auto;

        padding: 0 1rem 2rem 1rem;

    }

    section{

        width: 100%;

```

```
}

.comments-section,

.comments-add-section {

max-width: 100%;

}


.comments-add-section button {

width: auto;

max-width: 200px;

align-self: flex-start;

}

}

@media screen and (max-width: 480px) {

    nav {

        justify-content: center;

        gap: 1.5rem;

    }

    .comments-section,

    .comments-add-section {

padding: 0.8rem;

    }

    .comments-section div {
```

```
padding: 0.6rem;

}

.comments-add-section textarea {

min-height: 80px;

padding: 0.6rem;

}

.comments-add-section button {

padding: 0.7rem;

}

}
```

Função JavaScript que mostra os comentários dinamicamente na tela:

```
async function showComments() {

    const comments = await
get_func(`https://localhost:7120/api/Site/comments/${eventId}`)

    comments.map((comment) => {

const div = document.createElement("div")

const h3 = document.createElement("h3")

const p = document.createElement("p")
```

```

    async function getUsers() {

        const data = await
get_func(`https://localhost:7120/api/Users/${comment.userId}`)

        return data

    }

    getUsers().then((data) => {

        h3.textContent = data.name

        p.textContent = comment.commentary

        div.appendChild(h3)

        div.appendChild(p)

        commentsSection.appendChild(div)

    })

    })

}

showComments()

```

Print da página:

COMENTÁRIOS

DAVI FELL
OPORTUNIDADE IMPERDÍVEL

JORGE HENRIQUE
ÓTIMO

FABIANO CALVACANTE
MAL POSSO ESPERAR.

ALAN SANTOS
ESPERO QUE SEJA BOM.

LIN XI
INCRÍVEL

LEAN KING
VAI SER MARAVILHOSO

FABIANO CALVACANTE
PARECE MARAVILHOSO.

ADICIONE UM COMENTÁRIO

ADICIONAR

Tela em um dispositivo móvel:

LOGIN/CADASTRO

EVENTOS REGISTRADOS

COMENTÁRIOS

DAVI FELL
OPORTUNIDADE IMPERDÍVEL

ALAN SANTOS
ESPERO QUE SEJA BOM.

JORGE HENRIQUE
ÓTIMO

LIN XI
INCRÍVEL

LEAN KING
VAI SER MARAVILHOSO

FABIANO CALVACANTE
MAL POSSO ESPERAR.

FABIANO CALVACANTE
PARECE MARAVILHOSO.

ADICIONE UM COMENTÁRIO

ADICIONAR

2.3 HTML e CSS da página de login e cadastro:

HTML da página de login:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <title>Document</title>

    <script type="module" src="js/script.js"></script>

    <link rel="stylesheet" href="../../global.css">

    <link rel="stylesheet" href="login.css">

</head>

<body>

    <nav>

    <p>

        <a href="login.html">Login/Cadastro</a>

    </p>

    <p>

        <a href="../../EventPage/index.html">Eventos
Registrados</a>

    </p>
```

```

</nav>

<main>

<section class="cadastro-container">

<p>

        Ainda não tem cadastro em nosso site? Aperte o botão
abaixo para se cadastrar!

</p>

<button>

        <a href="../Cadastro/cadastro.html">Cadastro</a>

</button>

</section>

<section class="login-container">

        <p id="user-message"></p>

        <form class="login-form">

                <label for="userEmail">Email: </label>

                <input type="email" name="userEmail" id="userEmail">

                <label for="userPassword">Senha: </label>

                <input type="password" name="userPassword"
id="userPassword">

        </form>

        <button id="login-btn">Logar</button>

</section>

</main>

```

```
<footer>

<p>Site para registro e criação de eventos tecnológicos.</p>

<p>Feito por: </p>

<p>Vitória Freitas</p>

</footer>

</body>

</html>
```

CSS da página de login:

```
body{

    position: relative;

    height: 100vh;

    background-image: url("../img/world_img.png");

    background-position: right top ;

    background-repeat: no-repeat;

    background-attachment: fixed;

    min-height: 100vh;

    padding-bottom: 2rem;

}

main{

    position: absolute;

    top: 50%;
```

```

    left: 50%;

    transform: translate(-50%, -50%);

    display: flex;

    justify-content: center;

    width: 50vw;
}

section{

    height: 50vh;

    padding: 1rem;

    display: flex;

    flex-direction: column;

    justify-content: center;

    align-items: center;

    text-align: center;
}

.cadastro-container{

    background-color: #b4b4b4bb;

    width: 40%;

    border-radius: 0.5rem 0 0 0.5rem;
}

.login-container{

    background-color: #f3f3f3c9;

```

```

        width: 60%;

        border-radius: 0 0.5rem 0.5rem 0;
    }

    .login-form{

        display: flex;

        flex-direction: column;
    }

    .login-form input{

        width: 25vw;
    }

    button{

        width: 50%;

        margin-top: 1rem;
    }

    button:hover{

        text-decoration: underline;
    }

    a{

        text-decoration: none;

        color: #fff;
    }

    @media screen and (max-width: 500px) {

```

```

main{

width: 90vw;

}

button{

width: 70%;

}

}

```

Registro de usuário dinâmico com JavaScript:

```

inputSubmit.addEventListener("click", async (e) => {

    e.preventDefault()

    const userData = {

        Email: userEmail.value,

        Password: userPassword.value

    }

    messageToUser.innerText = "Carregando..."

    const res = await UserFetch(userData,
"https://localhost:7120/api/Users/Login")

```

```

messageToUser.innerText = ""

if(res.id){

// chave para saber se usuário está logado.

const keyUser = key + res.id

localStorage.setItem("key", keyUser)

window.location.assign(`/site/UserPage/index.html?id=${res.id}`)

}

else{

messageToUser.innerText = res

}

})

```

HTML da página de cadastro:

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta          name="viewport"          content="width=device-width,
initial-scale=1.0">

    <title>Document</title>

```

```

<script type="module" src="js/script.js"></script>

<link rel="stylesheet" href="../../global.css">

<link rel="stylesheet" href="cadastro.css">
</head>

<body>

    <nav>

        <p><a href="../../Login/login.html">Login/Cadastro</a></p>

        <p><a href="../../EventPage/index.html">Eventos
Registrados</a></p>

    </nav>

    <main>

        <p id="user-message"></p>

        <form >

            <label for="userName">Nome: </label>

            <input type="text" name="userName" id="userName">

            <label for="userAge">Idade: </label>

            <input type="number" name="userAge" id="userAge">

            <label for="userEmail">Email: </label>

            <input type="email" name="userEmail" id="userEmail">

            <label for="userPassword">Senha: </label>

            <input type="password" name="userPassword"
id="userPassword">

            <input type="submit" value="Finalizar Cadastro"
id="cadastro-btn">

```

```
</form>

</main>

<footer>

<p>Site para registro e criação de eventos tecnológicos.</p>

<p>Feito por: </p>

<p>Vitória Freitas</p>

</footer>

</body>

</html>
```

CSS da página de cadastro:

```
body{

    position: relative;

    height: 100vh;

}

main{

    position: absolute;

    top: 50%;

    left: 50%;

    transform: translate(-50%, -50%);

    display: flex;

    justify-content: center;
```

```
width: 70vw;

height: 50vh;

border: 1px solid rgba(124, 124, 124, 0.658);

border-radius: 0.5rem;
}

form{

display: flex;

flex-direction: column;

align-items: center;

justify-content: center;

width: 100%;

height: 100%;

padding: 10px;

margin: 1rem;
}

form input{

width: 60%;

margin: 0.5rem;

padding: 0.1rem;
}

form input[type="submit"] {

margin-right: 0.5rem;
```

```
padding: 0.5rem 1rem;

cursor: pointer;

border: none;

border-radius: 4px;

background-color: #6e6e6e;

color: white;

width: 30%;

margin: 1rem;
}

form label{

margin: 0.2rem;

}

@media screen and (max-width: 500px) {

main{

width: 80vw;

height: 60vh;

}

form input[type="submit"] {

width: 60%;

}

form input{
```

```
width: 100%;

margin: 0.9rem;

}

}
```

Cadastro de usuário com JavaScript:

```
cadastroBtn.addEventListener("click", async (e) => {

    e.preventDefault()

    const user = {

        Name: userName.value,

        Age: userAge.value,

        Email: userEmail.value,

        Password: userPassword.value

    }

    messageToUser.innerText = "Carregando..."

    const res = await UserFetch(user,
"https://localhost:7120/api/Users/Register")

    messageToUser.innerText = ""
```

```

    if(res.id){

        // chave para saber se usuário esta logado.

        const keyUser = key + res.id

        localStorage.setItem("key", keyUser)

        window.location.assign(`/site/UserPage/index.html?id=${res.id}`)

    }

    else{

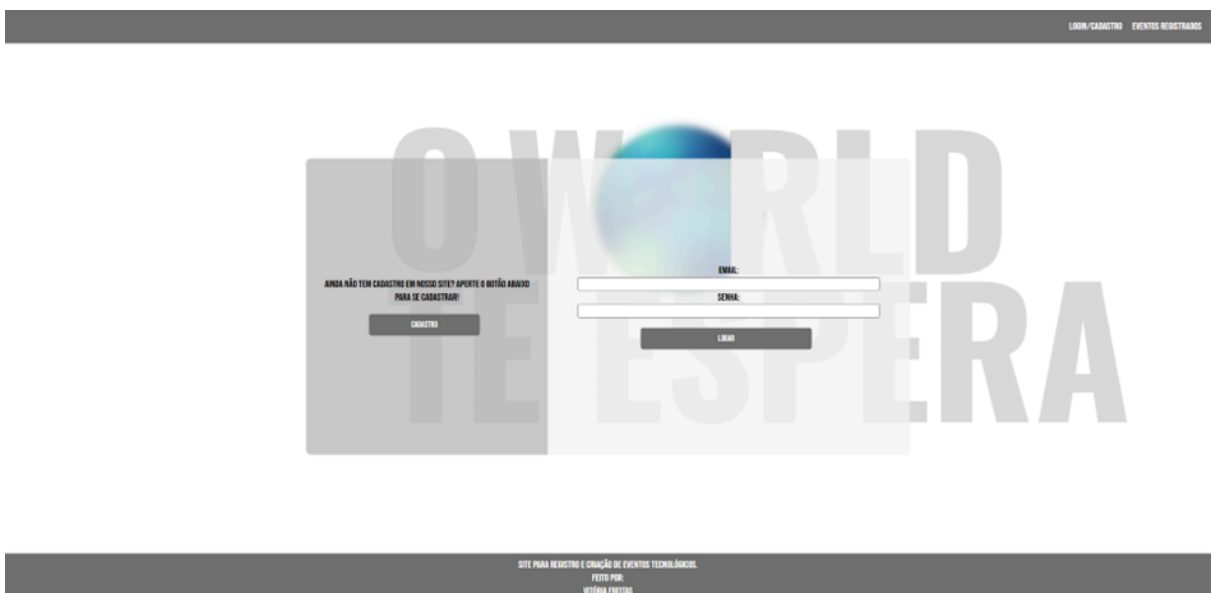
        messageToUser.innerText = res

    }

})

```

Prints das telas de login em computador e a cadastro em um celular, respectivamente:



NOME:

IDADE:

EMAIL:

SENHA:

FINALIZAR CADASTRO

2.4 HTML e CSS da página do usuário e da emissão de certificado:

HTML da página do usuário:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <title>Document</title>

    <script type="module" src="js/script.js"></script>
```

```

    <link rel="stylesheet" href="index.css">

    <link rel="stylesheet" href="../global.css">

</head>

<body>

    <header><button id="certificateBtn">Gerar certificado de um
evento</button></header>

    <main>

    <section class="form-event-container">

        <h2>Deseja registrar um evento? Preencha o formulário
abaixo: </h2>

        <p id="userMessage"></p>

        <form>

            <label for="title">Título: </label>

            <input type="text" name="title" id="title">

            <label for="description">Descrição: </label>

                <input type="text" name="description"
id="description">

            <label for="date">Data: </label>

            <input type="date" name="date" id="date">

                <label for="key">Key para emissão de certificação:
</label>

                <input type="text" name="key" id="key">

            <button id="addEventBtn">Adicionar evento</button>

        </form>

```

```

</section>

<section class="user-events-container">

    <div class="event-edit">

        <form >

            <h2>Edição de evento:</h2>

            <label for="titleEdit">Título: </label>

                <input type="text" name="titleEdit"
id="titleEdit">

            <label for="descriptionEdit">Descrição: </label>

                <input type="text" name="descriptionEdit"
id="descriptionEdit">

            <label for="dateEdit">Data: </label>

                <input type="date" name="dateEdit" id="dateEdit">

            <label for="keyEdit">Key para emissão de
certificacao: </label>

                <input type="text" name="keyEdit" id="keyEdit">

            <button id="EditEventBtn">Editar evento</button>

        </form>

    </div>

    <h2>Seus eventos registrados:</h2>

</section>

</main>

<footer>

<p>Site para registro e criação de eventos tecnológicos.</p>

```

```
<p>Feito por: </p>

<p>Vitória Freitas</p>

</footer>

</body>

</html>
```

CSS da página do usuário:

```
body{

    display: flex;

    flex-direction: column;

    align-items: center;

    margin-bottom: 5rem;

}

header{

    top: 0;

    width: 100%;

    display: flex;

    gap: 1rem;

    justify-content: flex-end;

    padding: 1rem;

    background-color: #6e6e6e;

}
```

```

header button{

    background-color: #fff;

    color: #3a3a3a;

}

header button:hover{

    text-decoration: underline;

}

main{

    border: 1px solid rgba(194, 194, 194, 0.685);

    border-radius: 0.5rem;

    margin: 2rem;

    padding: 0.8rem;

}

.form-event-container form{

    display: flex;

    flex-direction: column;

    padding: 1rem;

}

.form-event-container form button{

    margin: 0.8rem;

}

.user-events-container h2{

```

```
        text-align: center;

        margin-bottom: 0.2rem;
    }

    .event-user-container{

        border: 1px solid rgba(194, 194, 194, 0.685);

        border-radius: 0.5rem;

        padding: 0.8rem;

        margin: 0.5rem;
    }

    .event-user-container p {

        margin-top: 10px;
    }

    .event-user-container button{

        margin-top: 10px;
    }

    .event-edit{

        display: none;
    }

    .event-edit form{

        display: flex;

        flex-direction: column;

        align-items: center;
```

```

        margin: 0 auto;
    }

    .event-edit form input, label{

        margin: 0.1rem;
    }

    .event-edit form input{

        width: 100%;
    }

    .event-edit form button{

        margin: 0.5rem;
    }

```

Funções em JavaScript para pegarem os dados dos usuário, dados dos eventos do usuário. Também adicionar, deletar ou editar eventos.

```

async function loadUsers(idEvent, container){

    const users = await
    get_func(`https://localhost:7120/api/Site/registeredUsersInEvent/${idEvent}`)

    if(users){

        users.map((user) => {

            const name = document.createElement("p")

            const age = document.createElement("p")

            const email = document.createElement("p")

```

```

        name.textContent = user.name

        age.textContent = user.age

        email.textContent = user.email

        container.appendChild(name)

        container.appendChild(age)

        container.appendChild(email)

    })

    } else

    {

        userMessage.textContent = "Erro ao pegar os usuários."

    }

}

async function loadUserEvents(){

    const events = await
get_func(`https://localhost:7120/api/Events/getEventByUser/${userId}`)

    events.map((events) => {

        const div = document.createElement("div")

        const h3Title = document.createElement("h3")

        const pDesc = document.createElement("p")

        const pData = document.createElement("p")

        const pKey = document.createElement("p")

        const buttonEdit = document.createElement("button")

```

```
const buttonDelete = document.createElement("button")

const buttonShowUsers = document.createElement("button")

const date = new Date(events.date)

h3Title.textContent = `Título: ${events.title}`

pDesc.textContent = `Descrição: ${events.description}`

pData.textContent = `Data: ${date.toLocaleDateString("en-GB")}`

pKey.textContent = `Key: ${events.key}`

buttonEdit.textContent = "Editar"

buttonDelete.textContent = "Deletar"

buttonShowUsers.textContent = "Mostrar usuários inscritos"

div.classList.add("events-user-container")


div.appendChild(h3Title)

div.appendChild(pDesc)

div.appendChild(pData)

div.appendChild(pKey)

div.appendChild(buttonEdit)

div.appendChild(buttonDelete)

div.appendChild(buttonShowUsers)


div.classList.add("event-user-container")
```

```

userEventsContainer.appendChild(div)

// edição de dados

buttonEdit.addEventListener("click", async (e) => {

    e.preventDefault()

    eventEdit.style.display = "flex"

    eventId = events.id

})

buttonDelete.addEventListener("click", async (e) => {

    e.preventDefault()

    const data = {

        eventId: events.id,

        userId: userId

    }

    const res = await UserFetch(data,
"https://localhost:7120/api/Events/DeleteEvent", "delete")

    userMessage.textContent = res !== null ? "Deletado com
sucesso" : "Aconteceu um erro, tente novamente."

    window.location.reload()

})

buttonShowUsers.addEventListener("click", async (e) => {

    e.preventDefault()

    await loadUsers(events.id, div)

})

```

```

    })
}

addEventBtn.addEventListener("click", async (e) => {

    e.preventDefault()

    const data = {

        title: title.value,

        description: description.value,

        date: date.value,

        key: key.value,

        user_Id: userId

    }

    userMessage.textContent = "Carregando..."

    const res = await UserFetch(data,
"https://localhost:7120/api/Events/AddEvent")

    userMessage.textContent = res !== null ? "Adicionado com
sucesso" : "Aconteceu um erro, tente novamente."

    window.location.reload()

})

EditEventBtn.addEventListener("click", async (e) => {

    e.preventDefault()

    const data = {

        id: eventId,

        title: titleEdit.value,

```

```

        description: descriptionEdit.value,

        date: dateEdit.value,

        key: keyEdit.value,

        user_Id: userId

    }

    const res = await UserFetch(data,
"https://localhost:7120/api/Events/EditEvent", "put")

    userMessage.textContent = res !== null ? "Atualizado com
sucesso" : "Aconteceu um erro, tente novamente."

    window.location.reload()

})

```

Certificado HTML e CSS:

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <title>Document</title>

    <script type="module" src="js/certificado.js"></script>

    <link rel="stylesheet" href="../global.css">

    <link rel="stylesheet" href="certificado.css">

```

```

</head>

<body>

    <nav>

        <p><a
href="../Login_Cadastro/Login/login.html">Login/Cadastro</a></p>

        <p><a href="../EventPage/index.html">Eventos Registrados</a></p>

    </nav>

    <main>

        <h1>Insira a chave para gerar o certificado</h1>

        <label for="keyPass">Chave de acesso: </label>

        <input type="text" name="keyPass" id="keyPass">

        <button id="certificateBtn">Gerar certificado</button>

    </main>

    <footer>

        <p>Site para registro e criação de eventos tecnológicos.</p>

        <p>Feito por: </p>

        <p>Vitória Freitas</p>

    </footer>

</body>

</html>

```

```
body{
```

```
position: relative;

height: 100vh;

background-image: url("../img/img.png"), url("../img/img2.png");

background-position: calc(100% - 20px) 90%, calc(5% - 20px)
10%;

background-repeat: no-repeat;

background-attachment: fixed;

min-height: 100vh;

padding-bottom: 2rem;
}

main{

position: absolute;

top: 50%;

left: 50%;

transform: translate(-50%, -50%);

display: flex;

justify-content: center;

align-items: center;

width: 50vw;

height: 50vh;

flex-direction: column;

background-color: rgba(167, 167, 167, 0.507);

border-radius: 0.5rem;
```

```
}

main button{

    margin-top: 1rem;

    width: 30%;

}

main input{

    width: 50%;

    padding: 0.5rem;

    border: none;

    border-radius: 0.5rem;

}

main h1, label{

    text-align: center;

    margin: 0.3rem;

}

@media screen and (max-width: 500px) {

    main{

        width: 80vw;

    }

    main h1{

        font-size: 24px;

    }

}
```

```
}
```

Script em JavaScript que faz a emissão de certificado:

```
async function baixarImagemComDadosAPI(dados) {

    try {

        // Criar canvas

        const canvas = document.createElement('canvas');

        canvas.width = 600;

        canvas.height = 400;

        const ctx = canvas.getContext('2d');

        // Desenhar conteúdo

        ctx.fillStyle = '#2b2a2a';

        ctx.fillRect(0, 0, canvas.width, canvas.height);

        ctx.fillStyle = '#f8f8f8';

        ctx.font = 'bold 20px Arial';

        ctx.fillText(dados.eventTitle, 20, 50);

        const date = new Date(dados.date)

        ctx.font = '16px Arial';

        ctx.fillText(`Nome: ${dados.userName}`, 20, 100);

        ctx.fillText(`Data:   ${date.toLocaleDateString("en-GB")}`, 20,
140);

        ctx.fillText(`Descrição: ${dados.description}`, 20, 180);
```

```

// Baixar como PNG

canvas.toBlob((blob) => {

    const anchor = document.createElement('a');

    const url = URL.createObjectURL(blob);

    anchor.href = url;

    anchor.download = 'certificado.png';

    anchor.click();

    URL.revokeObjectURL(url);

}, 'image/png');

} catch (erro) {

    console.error('Erro:', erro);

}

}

certificateBtn.addEventListener("click", async () => {

    const data = {

        userId: userId,

        key: key.value

    }

    const res = await UserFetch(data,
"https://localhost:7120/api/Events/GetCertificateData")

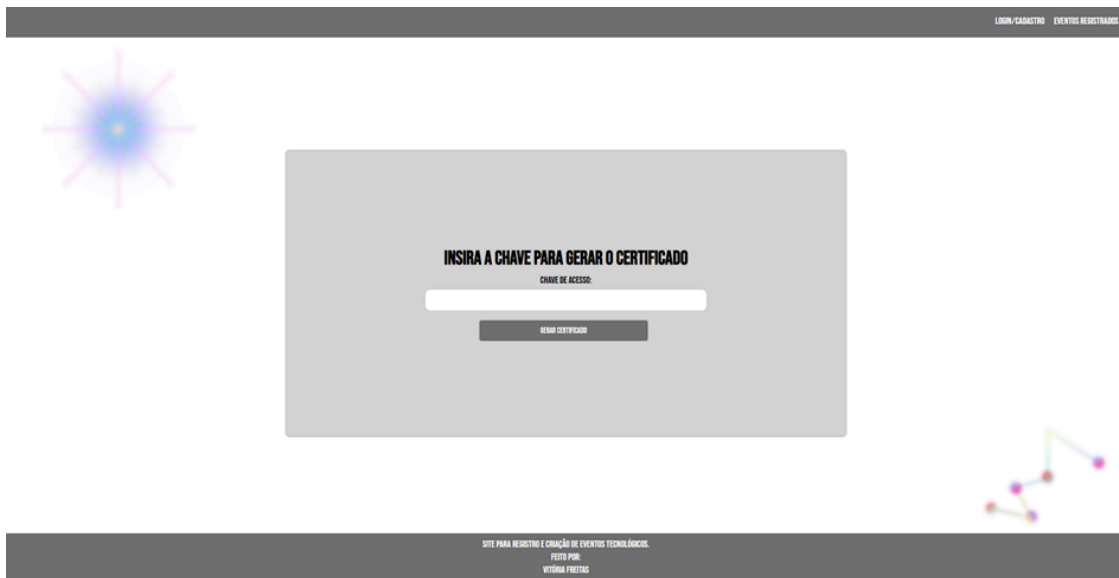
    console.log(res)

    if(res) baixarImagemComDadosAPI(res)

```

```
} )
```

Prints da tela de emissão de certificado em um computador e da tela do usuário em um celular, respectivamente:



CSS global da plataforma web:

```
@import
url('https://fonts.googleapis.com/css2?family=Bebas+Neue&display=swap'
);

*{

    font-family: "Bebas Neue", sans-serif;

    margin: 0;

    padding: 0;

    box-sizing: border-box;

}

nav {

    top: 0;

    position: fixed;

    width: 100%;

    display: flex;

    gap: 1rem;

    justify-content: flex-end;

    padding: 1rem;

    background-color: #6e6e6e;

}

nav a {

    text-decoration: none;

    color: #fff;
```

```
}

nav a:hover {

    text-decoration: underline;

}

button {

    margin-right: 0.5rem;

    padding: 0.5rem 1rem;

    cursor: pointer;

    border: none;

    border-radius: 4px;

    background-color: #6e6e6e;

    color: white;

}

footer{

    position: fixed;

    left: 0;

    bottom: 0;

    width: 100%;

    background-color: #6e6e6e;

    color: #fff;
```

```
padding: 0.5rem;

text-align: center;

}
```

3. Programação orientada a objetos com c#

3.1 Construção de API Web com ASP.NET Core para o desenvolvimento da aplicação web

Para a integração do front-end com o back-end da nossa aplicação web, desenvolvemos uma API, modelo RESTful usando a linguagem de programação C#, utilizando de tipagem de dados, classes, interfaces e herança.

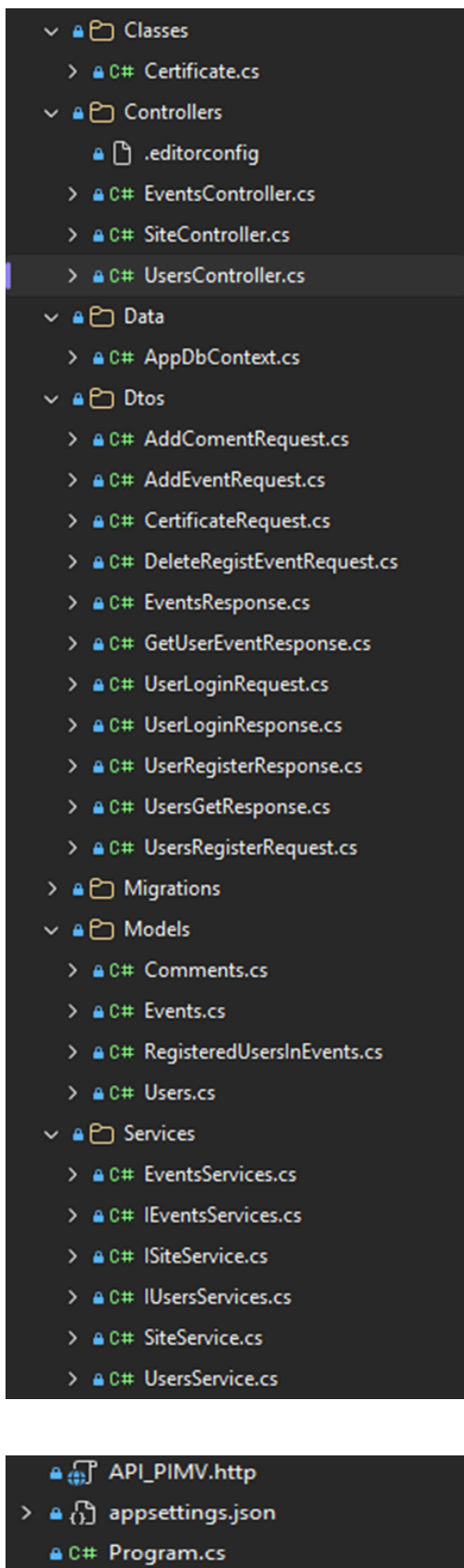
Uma API significa interface de programação de aplicativos, o modelo REST é o estilo arquitetural chamado Representational State Transfer, que é utilizado para conectar sistemas hipermídia distribuídos.

Uma API é um mecanismo que irá fazer a comunicação dos recursos dentro de um único sistema, esses recursos seriam por exemplo, o lado do cliente feito com JavaScript, HTML e CSS com o lado do servidor feito com C#, junto com o banco de dados SQL, que é o caso do nosso sistema web.

Para o nosso sistema incluímos diversas rotas na API, como para pegar os eventos na área da TI criados pelo os usuários, pegar um evento único, eventos criados por um único usuário, adicionar, editar ou deletar eventos, login, cadastro de usuários, dados para a emissão de certificado, registro de usuários em eventos, adicionar e mostrar comentários para comunicação entre usuários da plataforma e para pegar dados de um usuário ou de todos os cadastrados no sistema.

Decidimos que a estruturação API seria separada em pastas, assim separando funcionalidades e organizando o projeto de uma melhor forma, abaixo colocaremos uma imagem de como foi feita a organização do projeto. E logo em seguida começaremos mostrar as partes que julgamos mais importantes do nosso código

fonte para o desenvolvimento do back-end feito em C# com a IDE (Uma IDE é um ambiente para desenvolvimento integrado) Visual Studio Code.



Arquivo: Program.cs

```
using Microsoft.EntityFrameworkCore;

using API_PIMV.Services;

using Scalar.AspNetCore;

using API_PIMV.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddOpenApi();

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<IEventsServices, EventsServices>();

builder.Services.AddScoped<IUsersServices, UsersService>();

builder.Services.AddScoped<ISiteService, SiteService>();

builder.Services.AddCors(options =>
{
    options.AddPolicy("Allow",
    policy =>
    {
        policy.WithOrigins("http://127.0.0.1:5500")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();

    app.MapScalarApiReference();
}
```

```
app.UseHttpsRedirection();  
app.UseCors("Allow");  
app.UseAuthorization();  
app.MapControllers();  
app.Run();
```

3.2 Pasta com os Models do sistema

Arquivo: Comments.cs

```
namespace API_PIMV.Models  
{  
    public class Comments  
    {  
        public int Id { get; set; }  
        public string Commentary { get; set; }  
        public int UserId { get; set; }  
        public int EventId { get; set; }  
    }  
}
```

Arquivo: Events.cs

```
namespace API_PIMV.Models  
{  
    public class Events  
    {  
        public int Id { get; set; }  
    }  
}
```

```
public string Title { get; set; }

public string Description { get; set; }

public DateTime Date { get; set; }

public string Key { get; set; }

public int User_Id { get; set; }
}
}
```

Arquivo: RegisteredUsersInEvents.cs

```
namespace API_PIMV.Models
{
    public class RegisteredUsersInEvents
    {
        public int Id { get; set; }

        public int userId { get; set; }

        public int eventId { get; set; }
    }
}
```

Arquivo: Users.cs

```
namespace API_PIMV.Models
{
    public class Users
    {
        public int Id { get; set; }

        public string Name { get; set; }

        public int Age { get; set; }

        public string Email { get; set; }

        public string Password { get; set; }
    }
}
```

3.3 Pasta que contém uma classe do sistema

Arquivo: Certificate.cs

```
namespace API_PIMV.Classes
{
    public class Certificate
    {
        public string EventTitle { get; set; }

        public string UserName { get; set; }

        public string Description { get; set; }
    }
}
```

```

        public DateTime Date { get; set; }

    }
}

```

3.4 Pasta com os services do sistema

Arquivo: IEventService.cs

```

using API_PIMV.Classes;
using API_PIMV.Dtos;
using API_PIMV.Models;
namespace API_PIMV.Services
{
    public interface IEventsServices
    {
        Task<List<EventsResponse>> GetEvents();
        Task<EventsResponse?> GetEventById(int eventId);
        Task<List<GetUserEventResponse?>> GetEventsRegisteredByUser(int
userId);
        Task<Events> AddEvent(AddEventRequest eventObj);
        Task<Events> EditEvent(Events events);
        Task<bool> DeleteEvent(DeleteRegistEventRequest request);
        Task<Certificate> Certificate(CertificateRequest certificateBody);
    }
}

```

Arquivo: EventService.cs

```

using API_PIMV.Classes;

using API_PIMV.Data;

using API_PIMV.Dtos;

using API_PIMV.Models;

using Microsoft.EntityFrameworkCore;

namespace API_PIMV.Services
{
    public class EventsServices(AppDbContext context) : IEventsServices
    {
        public async Task<List<EventsResponse>> GetEvents()
        {
            var events = await context.Events.Select(c => new EventsResponse
            {
                Id = c.Id,
                Title = c.Title,
                Description = c.Description,
                Date = c.Date,
                User_Id = c.User_Id
            })
            .ToListAsync();

            if (events == null || events.Count == 0) throw new Exception("Eventos não encontrados.");

            return events;
        }

        public async Task<EventsResponse?> GetEventById(int eventId)
        {
            var eventFind = await context.Events

```

```

        .Where(c => c.Id == eventId)

        .Select(c => new EventsResponse
        {
            Id = c.Id,
            Title = c.Title,
            Date = c.Date,
            Description = c.Description,
            User_Id = c.User_Id
        })

        .FirstOrDefaultAsync();

        if (eventFind == null) throw new Exception("Evento não encontrado.");

        return eventFind;
    }

    public async Task<List<GetUserEventResponse?>> GetEventsRegisteredByUser(int
userId)
    {
        var events = await context.Events.Select(c => new Events
        {
            Id = c.Id,
            Title = c.Title,
            Description = c.Description,
            Date = c.Date,
            User_Id = c.User_Id,
            Key = c.Key
        })

        .ToListAsync();
    }

```

```

var userEvents = events

.Where(e => e.User_Id == userId)

.Select(e => new GetUserEventResponse()

{
    Id = e.Id,
    Title = e.Title,
    Description = e.Description,
    Date = e.Date,
    User_Id = e.User_Id,
    Key = e.Key
})

.ToList();

if (userEvents == null || userEvents.Count() == 0) throw new Exception("Evento não
encontrado.");

return userEvents;
}

public async Task<Events> AddEvent(AddEventRequest eventObj)
{
    var user = await context.Users.FindAsync(eventObj.User_Id);

    var eventFind = await context.Events.Where(c => c.Key ==
eventObj.Key).FirstOrDefaultAsync();

    if(eventFind != null) throw new Exception("Essa chave já existe.");

    if (user == null) throw new Exception("Usuário não encontrado.");

```

```

var newEvent = new Events()

{
    Title = eventObj.Title,
        Description = eventObj.Description,
    Date = eventObj.Date,
    User_Id = eventObj.User_Id,
    Key = eventObj.Key
};

context.Events.Add(newEvent);

await context.SaveChangesAsync();

return new Events()
{
    Id = newEvent.Id,
    Title = newEvent.Title,
        Description = newEvent.Description,
    Date = newEvent.Date,
    User_Id = newEvent.User_Id,
    Key = newEvent.Key
};
}

public async Task<Events> EditEvent(Events events)
{
    var eventFind = await context.Events.FindAsync(events.Id);

    if(eventFind == null) throw new Exception("Evento não encontrado.");

    if(eventFind.User_Id != events.User_Id) throw new Exception("Esse evento pode

```

apenas ser editado pelo usuário que o criou.");

```
eventFind.Title = events.Title;
```

```
eventFind.Description = events.Description;
```

```
eventFind.Date = events.Date;
```

```
eventFind.Key = events.Key;
```

```
await context.SaveChangesAsync();
```

```
return eventFind;
```

```
}
```

```
public async Task<bool> DeleteEvent(DeleteRegistEventRequest request)
```

```
{
```

```
var eventFind = await context.Events.FindAsync(request.eventId);
```

```
//Removendo da tabela de usuários inscritos também
```

```
await context.RegisteredUsersInEvents
```

```
.Where(c => c.eventId == request.eventId)
```

```
.ExecuteDeleteAsync();
```

```
if (eventFind == null) throw new Exception("Evento não encontrado.");
```

```
if (eventFind.User_Id != request.userId) throw new Exception("Esse evento pode  
apenas ser deletado pelo usuário que o criou.");
```

```
context.Events.Remove(eventFind);
```

```
await context.SaveChangesAsync();
```

```

        return true;
    }

    public async Task<Certificate> Certificate(CertificateRequest certificateBody)
    {
        var user = await context.Users.FindAsync(certificateBody.userId);

        if (user == null) throw new Exception("Usuário não encontrado.");

        var eventFind = await context.Events.Where(c => c.Key ==
certificateBody.key).FirstAsync();

        if (eventFind == null) throw new Exception("Evento não encontrado.");

        var certificate = new Certificate()
        {
            EventTitle = eventFind.Title,
            UserName = user.Name,
            Description = eventFind.Description,
            Date = eventFind.Date
        };

        return certificate;
    }
}

```

Arquivo: IUsersService.cs

```

using API_PIMV.Models;
using API_PIMV.Dtos;
namespace API_PIMV.Services
{
    public interface IUsersServices
    {
        Task<List<UsersGetResponse>> GetAllUsers();
        Task<UsersGetResponse?> GetUsers(int id);
        Task<UserRegisterResponse> RegisterUser(UsersRegisterRequest user);
        Task<UserLoginResponse> LoginUser(UserLoginRequest user);
    }
}

```

Arquivo: UsersService.cs

```

using Microsoft.EntityFrameworkCore;
using API_PIMV.Data;
using API_PIMV.Models;
using API_PIMV.Dtos;
using static BCrypt.Net.BCrypt;
namespace API_PIMV.Services
{
    public class UsersService(AppDbContext context) : IUsersServices
    {
        public async Task<List<UsersGetResponse>> GetAllUsers()
        {
            var users = await context.Users.Select(c => new UsersGetResponse

```

```

    {
        Id = c.Id,
        Name = c.Name,
        Age = c.Age,
        Email = c.Email
    }).ToListAsync();

    if (users == null || users.Count == 0) throw new Exception("Usuários não encontrados");

    return users;

}

public async Task<UsersGetResponse?> GetUsers(int id)
{

    var user = await context.Users
        .Where(c => c.Id == id)
        .Select(c => new UsersGetResponse
        {
            Id = c.Id,
            Name = c.Name,
            Age = c.Age,
            Email = c.Email
        }).FirstOrDefaultAsync();

    if (user == null) throw new Exception("Usuário não encontrado.");
}

```

```

        return user;

    }

    public async Task<UserRegisterResponse> RegisterUser(UsersRegisterRequest
user)
    {
        var userExist = await context.Users
            .Where(c => c.Email == user.Email)
            .FirstOrDefaultAsync();

        if(userExist != null) throw new Exception("E-mail já existente, coloque outro");

        if(user.Age < 18) throw new Exception("Usuário menor de idade");

        var userPassword = user.Password;
        var hashPassword = HashPassword(userPassword, 12);
        var NewUser = new Users
        {
            Name = user.Name,
            Age = user.Age,
            Email = user.Email,
            Password = hashPassword
        };

        context.Users.Add(NewUser);
        await context.SaveChangesAsync();

        return new UserRegisterResponse

```

```

{
    Id = NewUser.Id,
    Name = NewUser.Name,
    Age = NewUser.Age,
    Email = NewUser.Email
};

}

public async Task<UserLoginResponse> LoginUser(UserLoginRequest user)
{
    var userFind = await context.Users
        .Where(c => c.Email == user.Email)
        .FirstOrDefaultAsync();

    if(userFind == null) throw new Exception("E-mail incorreto.");

    var passwordIsCorrect = Verify(user.Password, userFind.Password);

    if (passwordIsCorrect == false) throw new Exception("Senha incorreta.");

    UserLoginResponse requestReturn = new UserLoginResponse()
    {
        Id = userFind.Id,
        Name = userFind.Name,
        Email = userFind.Email
    };

    return requestReturn;
}

```

```

    }
    }
}

```

Arquivo: [ISiteService.cs](#)

```

using API_PIMV.Dtos;
using API_PIMV.Models;

namespace API_PIMV.Services
{
    public interface ISiteService
    {
        Task<bool> AddComent(AddComentRequest comment);
        Task<List<Comments>> ShowComments(int eventId);
        Task<bool> RegisterToEvent(DeleteRegistEventRequest request);
        Task<List<UsersGetResponse>> UsersReginterInTheEvent(int eventId);
    }
}

```

Arquivo: [SiteService.cs](#)

```

using Microsoft.EntityFrameworkCore;
using API_PIMV.Data;
using API_PIMV.Models;
using API_PIMV.Dtos;

namespace API_PIMV.Services
{
    public class SiteService(AppDbContext context) : ISiteService

```

```

{
    public async Task<bool> AddComent(AddComentRequest comment)
    {

        var user = await context.Users.FindAsync(comment.userId);

        if (user == null) throw new Exception("Usuário não registrado.");

        var events = await context.Events.FindAsync(comment.eventId);

        if (events == null) throw new Exception("Evento não registrado");

        Comments newComment = new Comments()
        {
            Commentary = comment.comment,
            UserId = comment.userId,
            EventId = comment.eventId,
        };

        context.Comments.Add(newComment);

        await context.SaveChangesAsync();

        return true;
    }

    public async Task<List<Comments>> ShowComments(int eventId)
    {

```

```

var comments = await context.Comments
    .Where(c => c.EventId == eventId)
    .Select(c => new Comments
    {
        Id = c.Id,
        Commentary = c.Commentary,
        UserId = c.UserId,
        EventId = c.EventId
    })
    .ToListAsync();

if(comments == null || comments.Count() == 0)
{
    throw new Exception("Evento sem comentários.");
}

return comments;
}

public async Task<bool> RegisterToEvent(DeleteRegistEventRequest request)
{
    var user = await context.Users.FindAsync(request.userId);

    if (user == null) throw new Exception("Usuário não registrado.");

    var events = await context.Events.FindAsync(request.eventId);

    if (events == null) throw new Exception("Evento não registrado");

    RegisteredUsersInEvents register = new RegisteredUsersInEvents()

```

```

{
    userId = request.userId,
    eventId = request.eventId,
};

context.RegisteredUsersInEvents.Add(register);

await context.SaveChangesAsync();

return true;
}

public async Task<List<UsersGetResponse>> UsersRegisterInTheEvent(int eventId)
{
    var events = await context.Events.FindAsync(eventId);

    if (events == null) throw new Exception("Evento não registrado");

    var registeredUsersInEvent = await context.RegisteredUsersInEvents
        .Where(c => c.eventId == eventId)
        .Select(c => c.userId)
        .ToListAsync();

    /* pegar os usuários que estão dentro de registeredUsersInEvent */

    var requestReturn = await context.Users
        .Where(c => registeredUsersInEvent.Contains(c.Id))
        .Select(c => new UsersGetResponse
    {

```

```

        Id = c.Id,
        Name = c.Name,
        Age = c.Age,
        Email = c.Email
    }).ToListAsync();

    if (requestReturn.Count() == 0 || requestReturn == null)
    {
        throw new Exception("Esse evento não tem ninguém registrado nele.");
    }

    return requestReturn;
}
}
}

```

3.5 Pasta com os Controllers do Sistema

Arquivo: EventsController.cs

```

using API_PIMV.Classes;
using API_PIMV.Dtos;
using API_PIMV.Models;
using API_PIMV.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
namespace API_PIMV.Controllers
{

```

```

[Route("api/[controller]")]
[ApiController]
public class EventsController(IEventsServices service) : Controller
{
    [HttpGet("getEvents")]
    public async Task<ActionResult<List<EventsResponse>>> GetAllEvents()
    {
        try
        {
            var requestRes = await service.GetEvents();

            return Ok(requestRes);
        }
        catch (Exception ex)
        {
            return BadRequest(ex.Message);
        }
    }

    [HttpGet("getEvent/{id}")]
    public async Task<ActionResult<EventsResponse?>> GetEvent(int Id)
    {
        try
        {
            var eventReturned = await service.GetEventById(Id);

            return Ok(eventReturned);
        }
        catch (Exception ex)
    }

```

```

    {
        return BadRequest(ex.Message);
    }
}

[HttpGet("getEventByUser/{userId}")]
public async Task<ActionResult<List<GetUserEventResponse?>>> GetUserEvent(int
userId)
{
    try
    {
        var eventReturned = await service.GetEventsRegisteredByUser(userId);

        return Ok(eventReturned);
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}

[HttpPost("AddEvent")]
public async Task<ActionResult<Events>> AddingEvent(AddEventRequest eventObj)
{
    try
    {
        var eventAdd = await service.AddEvent(eventObj);

        return CreatedAtAction(nameof(GetEvent), new { id = eventAdd.Id }, eventAdd);
    }
}

```

```

    }

    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}

[HttpPut("EditEvent")]
public async Task<ActionResult<Events>> EventEdit(Events events)
{
    try
    {
        var eventEdit = await service.EditEvent(events);

        return CreatedAtAction(nameof(GetEvent), new { id = eventEdit.Id }, eventEdit);
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}

[HttpDelete("DeleteEvent")]
public async Task<ActionResult<bool>> DelEvent(DeleteRegistEventRequest request)
{
    try
    {
        var eventDel = await service.DeleteEvent(request);

        return Ok(eventDel);
    }
}

```

```

    }

    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}

[HttpPost("GetCertificateData")]
public async Task<ActionResult<Certificate>> CertificateData(CertificateRequest
certificateBody)
{
    try
    {
        var certificate = await service.Certificate(certificateBody);

        return Ok(certificate);
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
}
}

```

Arquivo: SiteController.cs

```

using API_PIMV.Dtos;
using API_PIMV.Models;

```

```

using API_PIMV.Services;

using Microsoft.AspNetCore.Http;

using Microsoft.AspNetCore.Mvc;

namespace API_PIMV.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class SiteController(ISiteService service) : Controller
    {
        [HttpPost("addComment")]
        public async Task<ActionResult<bool>> AddingComments(AddComentRequest comment)
        {
            try
            {
                var requestReturn = await service.AddComent(comment);

                if (requestReturn == true)
                {
                    return Ok(requestReturn);
                }

                return BadRequest(false);
            }
            catch (Exception ex)
            {
                return BadRequest(ex.Message);
            }
        }
    }
}

```

```

[HttpGet("comments/{eventId}")]
public async Task<ActionResult<List<Comments>>> ShowingEventComments(int eventId)
{
    try
    {
        var requestReturn = await service.ShowComments(eventId);

        return Ok(requestReturn);
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}

[HttpPost("registerToEvent")]
public async Task<ActionResult<bool>> EventRegister(DeleteRegistEventRequest request)
{
    try
    {
        var requestReturn = await service.RegisterToEvent(request);

        return Ok(requestReturn);
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}

```

```

    }

    }

    [HttpGet("registeredUsersInEvent/{eventId}")]
    public async Task<ActionResult<List<UsersGetResponse>>>
getUsersRegisteredInEvent(int eventId)
    {
        try
        {
            var requestReturn = await service.UsersReginterInTheEvent(eventId);

            return Ok(requestReturn);
        }
        catch (Exception ex)
        {
            return BadRequest(ex.Message);
        }
    }
}
}

```

Arquivo: UsersController.cs

```

using API_PIMV.Dtos;
using API_PIMV.Models;
using API_PIMV.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

```

```

namespace API_PIMV.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UsersController(IUsersServices service) : Controller
    {
        [HttpGet]
        public async Task<ActionResult<List<UsersGetResponse>>> Getusers()
        {
            try
            {
                return Ok(await service.GetAllUsers());
            }
            catch (Exception ex)
            {
                return BadRequest(ex.Message);
            }
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<UsersGetResponse?>> GetUserById(int id)
        {
            try
            {
                var user = await service.GetUsers(id);

                return Ok(user);
            }
        }
    }
}

```

```

    }

    catch (Exception ex)
    {
        if(ex.Message == "Usuário não encontrado.")
        {
            return NotFound("User não encontrado.");
        }

        return BadRequest(ex.Message);
    }
}

[HttpPost("Register")]

public async Task<ActionResult<UserRegisterResponse>>
PostUser(UsersRegisterRequest user)
{
    try
    {
        var newUser = await service.RegisterUser(user);
        return CreatedAtAction(nameof(GetUserById), new { id = newUser.Id }, newUser);
    }

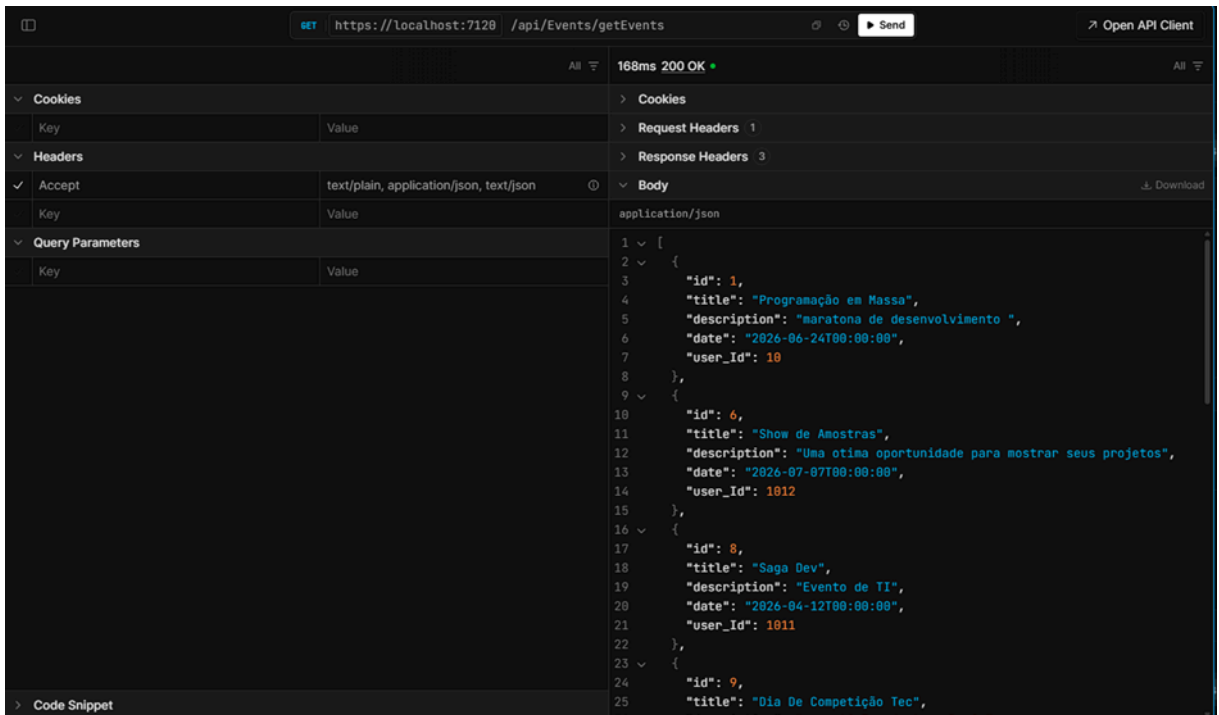
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}

```

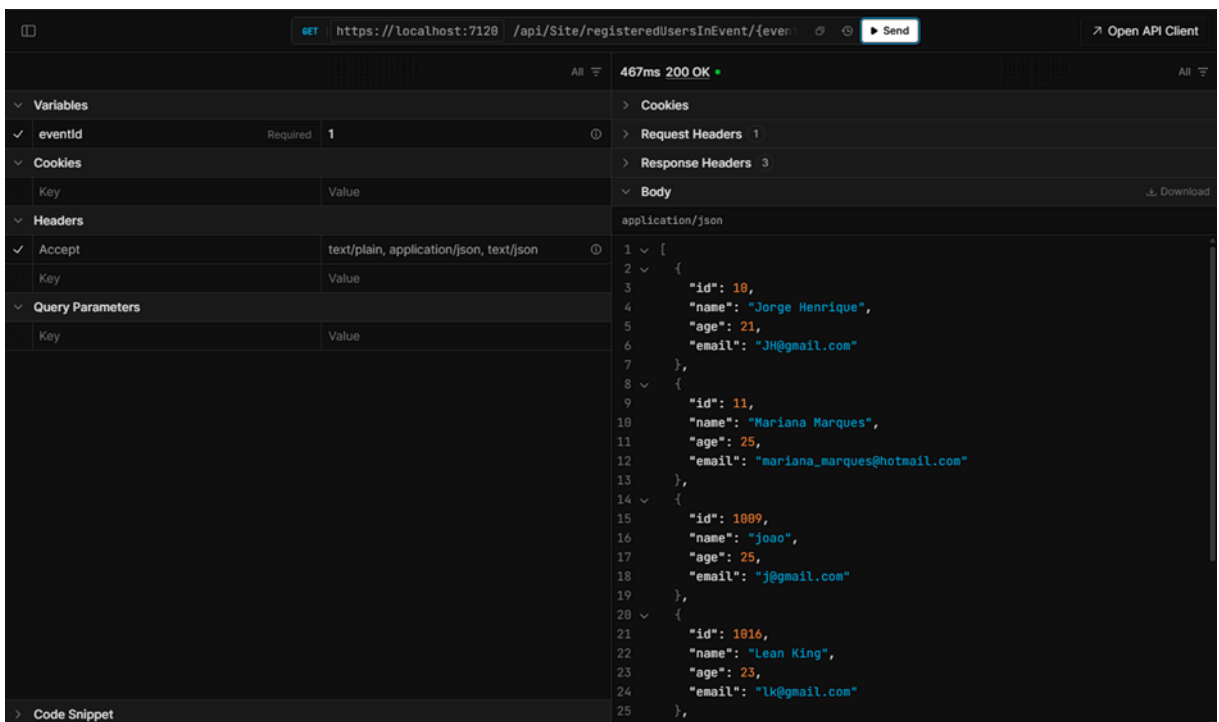
```
[HttpPost("Login")]
public async Task<ActionResult<UserLoginResponse>> Login(UserLoginRequest user)
{
    try
    {
        var requestReturn = await service.LoginUser(user);
        return Ok(requestReturn);
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
}
```

3.6 Prints de casos de testes

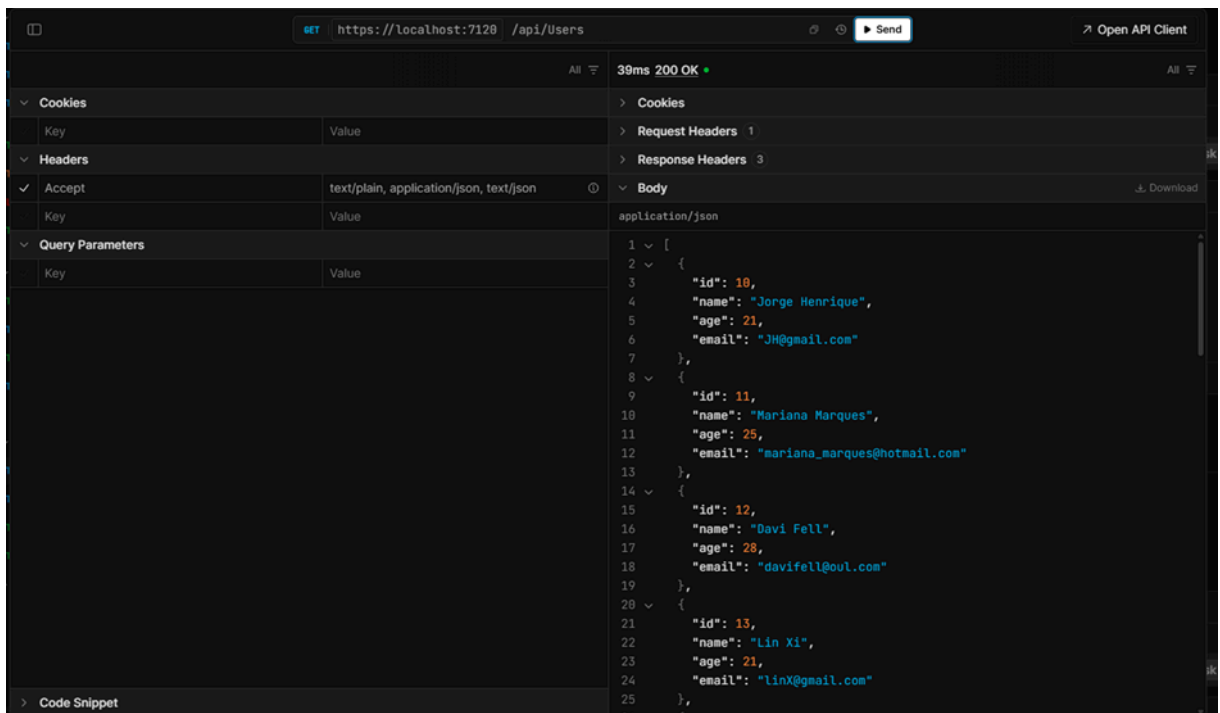
Caso de teste da rota: /api/Event/getEvents, tempo de resposta: 168 ms.



Caso de teste da rota: `/api/Site/registeredUsersInEvent/{eventId}`, tempo de resposta: 467 ms.



Caso de teste da rota: `/api/Users`, tempo de resposta: 30ms.



4. Comunicação, Liderança e Negociação no Projeto "Hello World Conference"

No desenvolvimento da plataforma "Hello World Conference", a comunicação, a liderança e a negociação assumem um papel estratégico para garantir a integração entre as áreas técnicas e humanas do projeto, contribuindo diretamente para a eficiência da gestão do evento e para a experiência dos usuários.

A comunicação organizacional, nesse contexto, é responsável por estruturar o fluxo de informações entre os diferentes públicos envolvidos no evento, como participantes, palestrantes, patrocinadores, coordenação acadêmica e equipe técnica. Para isso, foi elaborado um plano de comunicação que define objetivos específicos para cada público, bem como os canais mais adequados para cada tipo de mensagem.

No ambiente da plataforma, a comunicação ocorre de forma integrada por meio de diferentes recursos, como notificações no sistema, e-mails automatizados, páginas informativas e interações via comentários. Cada canal possui uma finalidade específica: os e-mails são utilizados para confirmações e avisos formais (como inscrição e emissão de certificados), enquanto a própria plataforma funciona como

um canal dinâmico para atualizações em tempo real, como mudanças na programação ou avisos importantes.

Como forma de aplicação prática do plano de comunicação, foi desenvolvido um modelo de e-mail em HTML com identidade visual moderna e alinhada à proposta tecnológica do sistema. O objetivo desse modelo é demonstrar como a comunicação pode ser aplicada de forma padronizada, atrativa e eficiente, incentivando a participação dos usuários por meio de elementos visuais e chamadas para ação.

A seguir, apresenta-se um exemplo de código utilizado para envio de convite aos participantes:

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
<meta charset="UTF-8">
<title>Hello World Conference</title>

<style>
  body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #0f0f1a, #1a1a2e);
    color: #ffffff;
  }

  .container {
    max-width: 600px;
    margin: 40px auto;
    background: #111827;
    border-radius: 12px;
    overflow: hidden;
    box-shadow: 0 0 25px rgba(0, 255, 170, 0.2);
  }

  .header {
    background: linear-gradient(90deg, #7f00ff, #00f5a0);
    padding: 30px;
    text-align: center;
    font-size: 26px;
    font-weight: bold;
  }

  .content {
    padding: 25px;
  }

  .evento {
    background: rgba(255,255,255,0.05);
    padding: 15px;
```

```

border-radius: 8px;
margin: 20px 0;
border-left: 4px solid #00f5a0;
}

.evento strong {
  color: #00f5a0;
}

.botao {
  display: inline-block;
  padding: 14px 25px;
  background: linear-gradient(90deg, #00f5a0, #00c3ff);
  color: #000;
  font-weight: bold;
  text-decoration: none;
  border-radius: 6px;
  transition: 0.3s;
}

.botao:hover {
  opacity: 0.8;
}

.footer {
  text-align: center;
  font-size: 12px;
  color: #aaa;
  padding: 15px;
  background: #0d0d15;
}
</style>
</head>
<body>
<div class="container">
  <div class="header">
    🚀 Hello World Conference
  </div>
  <div class="content">
    <p>Olá!</p>
    <p>
      Prepare-se para mergulhar no universo da tecnologia!
      O <strong>Hello World Conference</strong> chegou para conectar você com inovação,
      conhecimento e oportunidades.
    </p>
    <div class="evento">
      <strong>Evento:</strong> Programação em Massa<br>
      <strong>Data:</strong> 24/06/2026<br>
      <strong>Formato:</strong> Online
    </div>
  </div>

```

```

    <p>
      🔥 Participe de palestras incríveis, interaja com outros desenvolvedores e fortaleça sua
      jornada na área de TI.
    </p>

    <p style="text-align:center;">
      <a href="#" class="botao">GARANTIR MINHA VAGA</a>
    </p>

  </div>

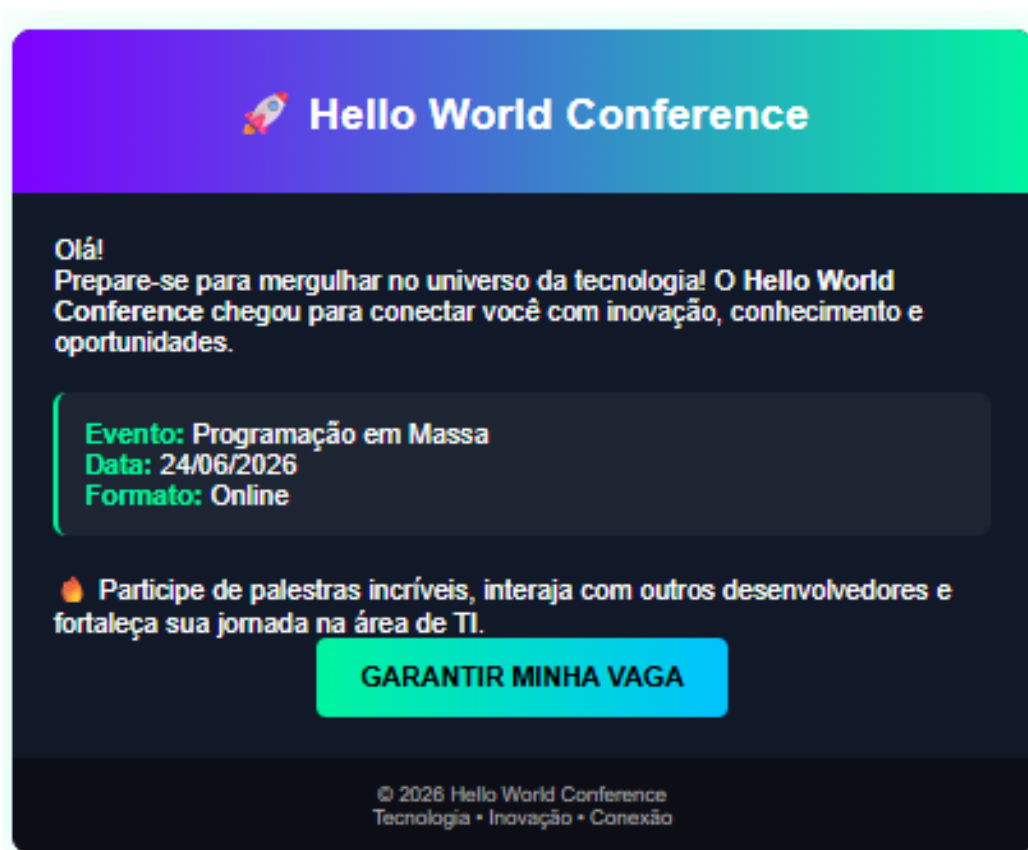
  <div class="footer">
    © 2026 Hello World Conference<br>
    Tecnologia • Inovação • Conexão
  </div>

</div>

</body>
</html>

```

Print do comunicado:



Além disso, as mensagens foram pensadas para serem claras, objetivas e acessíveis, evitando ambiguidades e garantindo que todos os usuários compreendam facilmente as informações. Considerando a proposta inclusiva do

projeto, a comunicação também deve contemplar acessibilidade, incluindo recursos em Libras e organização de conteúdos de forma visualmente compreensível, promovendo inclusão e equidade no acesso à informação.

A liderança dentro do projeto está diretamente ligada à coordenação das atividades e à tomada de decisões durante todas as etapas do desenvolvimento e execução do evento. O líder atua como facilitador, garantindo que a equipe esteja alinhada com os objetivos do projeto e que as demandas sejam atendidas dentro dos prazos estabelecidos.

Nesse sentido, adota-se um modelo de liderança participativa, no qual há incentivo à colaboração, troca de ideias e envolvimento dos membros da equipe nas decisões. Esse tipo de abordagem contribui para um ambiente mais produtivo e inovador, especialmente em projetos interdisciplinares como este, que envolvem tanto desenvolvimento técnico quanto planejamento organizacional.

O líder também é responsável por antecipar possíveis problemas, gerenciar conflitos e manter a motivação da equipe, assegurando que todos compreendam seu papel dentro do projeto. Dessa forma, a liderança não se limita à gestão de tarefas, mas também envolve aspectos humanos, como empatia, escuta ativa e valorização das contribuições individuais.

A negociação, por sua vez, é essencial para lidar com situações de conflito e alinhar interesses entre os diferentes envolvidos no evento. Em um ambiente acadêmico, é comum surgirem divergências relacionadas à programação, disponibilidade de palestrantes, critérios de certificação ou até mesmo demandas técnicas da plataforma.

Diante disso, são aplicadas estratégias de negociação baseadas no modelo integrativo, que busca soluções em que todas as partes sejam beneficiadas. A escuta ativa é um dos principais elementos nesse processo, permitindo compreender as necessidades de cada lado antes de propor alternativas.

Um exemplo prático de cenário de negociação no projeto é a disputa por horários de apresentação entre palestrantes. Nesse caso, a solução pode envolver a reorganização da programação ou a criação de sessões paralelas, garantindo que

todos sejam contemplados. Outro exemplo é o desacordo sobre a emissão de certificados, que pode ser resolvido por meio de comunicação transparente dos critérios e, se necessário, ajustes que atendam às expectativas sem comprometer a credibilidade do evento.

Além disso, a negociação também está presente na relação com os usuários da plataforma, especialmente na mediação de comentários e interações dentro do sistema, garantindo um ambiente respeitoso e colaborativo.

Dessa forma, a integração entre comunicação eficiente, liderança estratégica e negociação equilibrada fortalece a gestão do evento e contribui para o sucesso da plataforma "Hello World Conference". Esses elementos permitem não apenas a organização adequada das atividades, mas também a construção de uma experiência mais inclusiva, colaborativa e alinhada às necessidades dos participantes.

5. Língua Brasileira de Sinais – Libras

Nesta seção, apresenta-se o planejamento e a prototipagem dos recursos em língua de sinais integrados ao ambiente virtual responsivo, visando a inclusão plena de participantes surdos em eventos acadêmicos de TI. O projeto adota uma perspectiva bilíngue, reconhecendo a Libras como primeira língua (L1) e o Português escrito como segunda língua (L2).

5.1 Glossário de Termos Técnicos em Libras

Para garantir a compreensão técnica, foram selecionados termos recorrentes que possuem sinalização específica na plataforma:

- **Inscrição:** Termo essencial para o fluxo inicial do usuário.
- **Certificado:** Referente à emissão de documentos comprobatórios.
- **Palestra / Minicurso:** Categorias de atividades da programação.
- **Credenciamento / Apoio:** Instruções de acesso e suporte.

5.2 Storyboard dos Recursos de Acessibilidade

O desenvolvimento visual dos recursos de Libras foi estruturado para guiar o usuário surdo de forma intuitiva através da interface responsiva.

5.3 Storyboard de Orientação, Navegação e Apoio ao Participante Surdo



5.4 Análise Crítica e Perspectiva Bilíngue

A prototipagem apresentada na Figura X cumpre os requisitos de acessibilidade digital ao integrar o intérprete virtual em pontos estratégicos da jornada do usuário.

- **Roteirização:** Os vídeos de boas-vindas e tutoriais utilizam estruturas gramaticais próprias da Libras (tópico-comentário), evitando o "português sinalizado" para garantir a clareza comunicativa.
- **Interface Responsiva:** A janela do intérprete foi planejada para manter a visibilidade em diferentes dispositivos (smartphones e desktops), acompanhando o layout fluido em HTML5 e CSS3 desenvolvido no projeto.
- **Direito Linguístico:** A presença de recursos de apoio e suporte em Libras assegura que o participante surdo seja reconhecido como

sujeito de direito dentro do contexto acadêmico de TI, reduzindo barreiras de comunicação.

Conclusão

Ao longo do desenvolvimento deste trabalho, foi possível compreender a importância de integrar tecnologia e aspectos humanos na criação de soluções digitais. A plataforma "Hello World Conference" demonstrou ser uma proposta eficiente para a organização de eventos acadêmicos, reunindo funcionalidades práticas e uma interface acessível.

Além dos conhecimentos técnicos aplicados, o projeto também destacou a relevância da comunicação, da liderança e da negociação para o bom funcionamento do trabalho em equipe e para a resolução de desafios durante o processo. Outro ponto importante foi a preocupação com a acessibilidade, com a inclusão de recursos em Libras, ampliando o acesso de diferentes públicos.

Dessa forma, o projeto contribui para a criação de ambientes digitais mais organizados, inclusivos e alinhados às necessidades dos usuários, unindo tecnologia e inclusão de maneira eficiente.

Bibliografia

https://www-ibm-com.translate.google.com/translate?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

<https://aws.amazon.com/pt/what-is/ide/>

<https://us15.admin.mailchimp.com/>

https://www.grupounibra.com/repositorio/RHUMN/2021/a-importancia-da-comunicacao-dentro-da-organizacao2.pdf?utm_source=chatgpt.com

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: Informação e documentação: Trabalhos acadêmicos: Apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 10520: Informação e documentação: Citações em documentos: Apresentação. Rio de Janeiro: ABNT, 2023.

BRASIL. Lei nº 10.436, de 24 de abril de 2002. Dispõe sobre a Língua Brasileira de Sinais - Libras e dá outras providências. Brasília, DF, abr. 2002.

BRASIL. Decreto nº 5.626, de 22 de dezembro de 2005. Regulamenta a Lei nº 10.436, de 24 de abril de 2002, e o art. 18 da Lei nº 10.098, de 19 de dezembro de 2000. Brasília, DF, dez. 2005.